RAG.

30 QUESTIONS EVERY AI ENGINEER SHOULD ANSWER.

From your first chunk to your final answer — the **30 questions** that decide whether you actually understand retrieval-augmented generation.

N° 01

RAG FUNDAMENTALS

What is Retrieval Augmented Generation (RAG), and why is it critical for LLMs?

THE ANSWER

Retrieval Augmented Generation (RAG) is a technique used to upgrade Large Language Models (LLMs) by connecting them to external, private data.

Standard LLMs are limited because their knowledge is static (cutoff at their training date) and they cannot access private company data. RAG solves this by:

1. Retrieving up-to-date or proprietary information from an external source (like a database).
2. Injecting that information into the prompt.
3. Generating an answer based on that specific context.

N° 02

RAG ARCHITECTURE

What are the two main components of a RAG application architecture?

THE ANSWER

A RAG application is built on two distinct phases:

• Indexing (The Preparation Phase): This happens offline. Data is ingested, cleaned, and organized. It involves loading documents, splitting them into smaller chunks, and storing them in a vector database so they can be searched later.

• Retrieval & Generation (The Runtime Phase): This happens when a user asks a question. The system retrieves relevant chunks from the index and passes them to the LLM to generate a final answer.

N° 03

INDEXING

How does the indexing process work, and why is it essential?

THE ANSWER

Indexing is the process of organizing your data so the LLM can "read" it efficiently. It involves three specific steps:

1. Load: Importing data using Document Loaders (e.g., for PDFs or Webpages).
2. Split: Breaking large texts into smaller "chunks" using Text Splitters.
3. Store: Converting chunks into numbers (embeddings) and saving them in a VectorStore.

Without proper indexing, the system cannot search through data fast enough to answer in real-time.

N° 04

RETRIEVAL

What is the specific role of the "Retrieval" component?

THE ANSWER

The Retrieval component acts as the bridge between the user's question and the database. Its only job is to find the most relevant pieces of information (context) based on the user's query.

If the retrieval step fails and fetches irrelevant data, the LLM will likely hallucinate or provide a wrong answer. High-quality retrieval is the most important factor for accurate RAG outputs.

N° 05

CHUNKING

Why is it necessary to split documents into smaller chunks?

THE ANSWER

You cannot simply feed an entire 100-page document into an LLM at once. Splitting is required because:

- Context Limits: LLMs have a limit on how much text they can process (the "context window").
- Precision: Searching for a specific paragraph is more accurate than searching for a whole book.
- Efficiency: Smaller chunks are faster to process and cheaper to embed.

N° 06

OBSERVABILITY

Why is configuring LangSmith crucial for complex applications?

THE ANSWER

LangSmith is a tool for observability. When building complex chains, it is often difficult to see where an error occurred or why an agent gave a specific answer.

LangSmith allows developers to "trace" the application, providing a step-by-step log of exactly what inputs entered the model and what outputs came out. This is essential for debugging and optimizing performance.

N° 07

LANGCHAIN SETUP

What are the essential dependencies for a LangChain RAG setup?

THE ANSWER

To build a standard RAG pipeline, you need three core packages:

- langchain: The main framework for building the chain.
- langchain_community: Contains third-party integrations (like loaders and specific tools).
- langchain_chroma: Connects your app to the Chroma vector database.

Install via: pip install langchain langchain_community langchain_chroma

N° 08

DATA INGESTION

What are the roles of WebBaseLoader and RecursiveCharacterTextSplitter?

THE ANSWER

These two tools work together to ingest web data:

• WebBaseLoader: Scrapes the HTML from a specific URL and converts it into text format.
• RecursiveCharacterTextSplitter: Takes that text and intelligently chops it into smaller pieces. It is "recursive" because it tries to keep paragraphs and sentences intact rather than cutting sentences in half.

N° 09

CONFIGURATION

What do the environment variables LANGCHAIN_TRACING_V2 and LANGCHAIN_API_KEY do?

THE ANSWER

These variables are used to connect your local application to the LangSmith cloud platform:

• LANGCHAIN_TRACING_V2: A switch (set to "true") that turns on the logging/tracing feature.

• LANGCHAIN_API_KEY: Your secret password that authorizes your code to send logs to your specific LangSmith project.

N° 10

DOCUMENT LOADERS

What is the primary purpose of DocumentLoaders?

THE ANSWER

DocumentLoaders standardize incoming data. Whether your source is a text file, a website, or a CSV, the loader converts it into a list of Document objects.

Every Document object always contains two things:

1. page_content: The actual information/text.
2. metadata: Contextual details (like source URL or page number) which are useful for citing sources later.

N° 11

WEB SCRAPING

How does WebBaseLoader function within the LangChain ecosystem?

THE ANSWER

WebBaseLoader is the bridge between the internet and your application. It allows LangChain to access live websites by:

1. Fetching: Using urllib to download the raw HTML from a URL.
2. Parsing: Using BeautifulSoup to strip away HTML tags and extract the actual text.
3. Formatting: Converting that text into a list of Document objects (containing content and metadata) that the system can process.

N° 12

HTML PARSING

How can you customize the HTML parsing process in WebBaseLoader?

THE ANSWER

You can refine what the loader scrapes by passing arguments to the underlying parser via the bs_kwargs (BeautifulSoup keyword arguments) parameter.

This is often necessary because most websites contain "noise" like navigation bars, footer links, and ads. By configuring specific parameters (like using a SoupStrainer), you can instruct the loader to ignore the noise and only extract text from specific HTML classes.

N° 13

DATA CLEANING

What is the specific role of BeautifulSoup in this process?

THE ANSWER

BeautifulSoup is the engine responsible for cleaning the data. While WebBaseLoader connects to the site, BeautifulSoup navigates the complex HTML structure (the DOM). It filters out the code (tags like , <script>, <style>) and extracts the human-readable text, ensuring the data passed to the LLM is clean and usable.

N° 14

PARSING EFFICIENCY

What is a SoupStrainer and how does it improve efficiency?

THE ANSWER

A SoupStrainer is a specific filter used during the parsing phase. Instead of loading the entire webpage and then deleting what you don't need, a SoupStrainer tells the system to only parse specific tags from the very beginning.

For example, you can tell it to only look for div tags with the class main-content. This significantly speeds up processing and ensures irrelevant data is never even loaded into memory.

N° 15

DATA QUALITY

Why is customized HTML parsing critical for a RAG system?

THE ANSWER

The quality of your output depends on the quality of your input ("Garbage In, Garbage Out").

If you feed your RAG system headers, copyright notices, and advertisements, the retrieval system might confuse this "noise" for actual answers. Customizing the parsing ensures that only high-value content is indexed, which leads to more accurate and concise answers from the AI.

N° 16

CHUNKING STRATEGY

Why must long documents be split before embedding?

THE ANSWER

Splitting documents (chunking) is mandatory for two reasons:

1. Context Window Limits: LLMs have a fixed memory limit. You cannot fit a 300-page manual into a single prompt, but you can fit relevant paragraphs.
2. Search Precision: It is much easier to find a specific answer if the database is made of small, focused concepts rather than massive, unstructured documents.

N° 17

CHUNK OVERLAP

What is "chunk overlap" and why is it used?

THE ANSWER

Chunk overlap creates a "sliding window" effect where the end of one chunk is repeated at the beginning of the next (e.g., a 200-character overlap).

This is vital for preserving context. Without overlap, a sentence or idea might be cut in half at the split point, causing the model to lose the connection between two related statements. Overlap stitches the "seams" together so no meaning is lost.

N° 18

TEXT SPLITTING

Why is RecursiveCharacterTextSplitter recommended for generic text?

THE ANSWER

This splitter is "smart" about how it breaks text apart. Instead of blindly cutting text every 1,000 characters, it recursively tries to split by:

1. Paragraphs (double newlines)
2. Sentences (single newlines)
3. Spaces

This hierarchy ensures that semantically related text stays together. It avoids breaking a sentence in the middle, which helps the LLM understand the full meaning of the retrieved chunk.

N° 19

METADATA

How does the add_start_index=True parameter help?

THE ANSWER

This feature adds a "coordinate" to every chunk of text. It records the character position (start_index) where that chunk originally appeared in the source document.

This is incredibly useful for citations. It allows the application to trace an answer back to its exact location in the original file, helping users verify where the information came from.

N° 20

DOCUMENT PROCESSING

What is the output of the split_documents() method?

THE ANSWER

The split_documents() method outputs a Python list of Document objects.

Each object represents one "chunk" and contains:

- page_content: The slice of text.
- metadata: Information about the source, including the new start_index if configured.

This list is the final format required to begin the embedding and indexing process.

N° 21

EMBEDDINGS

What is the purpose of embedding text and storing it in a vector store?

THE ANSWER

The embedding process transforms human language into a format that computers can understand and search efficiently.

• Embedding: An embedding model converts a chunk of text into a long list of numbers (a vector) that represents the meaning of that text.

• Storing: These vectors are saved in a Vector Store (a specialized database).

This is crucial because we cannot perform fast semantic searches on raw text. By converting text to numbers, the system can mathematically calculate which documents are related to the user's question in milliseconds.

N° 22

VECTOR SEARCH

What is Cosine Similarity and why is it used in vector searches?

THE ANSWER

Cosine similarity is the ruler used to measure how "close" two pieces of text are in meaning.

When a user asks a question, it is converted into a vector. The system then calculates the angle (cosine) between the question vector and all the document vectors in the store.

- Small Angle: The documents are highly similar (relevant).
- Large Angle: The documents are unrelated.

It focuses on the direction of the vector (the semantic meaning) rather than the magnitude (length), making it highly effective for matching questions to answers.

N° 23

VECTOR STORE SETUP

What are the key components needed to build and query a vector store?

THE ANSWER

To get a vector store running in LangChain, you need four interacting parts:

1. Documents: The source material (chunked text).
2. Embedding Model: The translator that turns text into vectors.
3. Vector Database: The warehouse (e.g., Chroma, Pinecone) that stores the vectors.
4. Similarity Search: The function that queries the database to find the "nearest neighbors" to your question.

N° 24

CHROMA DB

What are common challenges when working with the Chroma vector store?

THE ANSWER

Developers often face three main hurdles:

• Data Formatting: Chroma requires strictly formatted Document objects. Passing raw strings or incorrectly structured lists will cause errors.

• API Key Management: Since the embedding step often uses external APIs (like OpenAI), missing or improperly set environment keys will crash the ingestion process.

• Parameter Confusion: Knowing the difference between search parameters (e.g., k for the number of results vs. specific distance thresholds) can be tricky but is vital for tuning results.

N° 25

DEBUGGING

How do you inspect the contents of a vector store?

THE ANSWER

Unlike a standard SQL database, you can't just "open" a vector store to read rows easily. To verify data is actually there, you typically use:

- _collection.count(): A method to return the total number of chunks stored.
- Dummy Queries: Running a similarity_search with a generic term to see what comes back.

These "sanity checks" are necessary to confirm that your ingestion pipeline actually worked and that the store isn't empty.

N° 26

INGESTION PIPELINE

What is the step-by-step workflow to embed and store documents?

THE ANSWER

The standard workflow involves four distinct stages:

1. Prepare: Convert your raw text chunks into the official Document object format.
2. Configure: Initialize your embedding model (ensure API keys are active).
3. Ingest: Use the from_documents() method. This automatically embeds the text and pushes the vectors into the Chroma store in one go.
4. Validate: Immediately run a test query to ensure the system returns relevant results.

N° 27

RETRIEVERS

What is the specific role of a "Retriever" in LangChain?

THE ANSWER

A Retriever is an interface (or wrapper) that creates a standard way to fetch data.

While a Vector Store is a database, a Retriever is the tool that interacts with that database. It takes a string (the user's question) as input and returns a list of Document objects. It abstracts away the complex math of vector search, allowing the rest of the application to just ask for "relevant documents" without worrying about how they are found.

N° 28

VECTOR RETRIEVAL

How does the VectorStoreRetriever function?

THE ANSWER

This is a specific type of retriever that wraps around a vector database.

- Input: It accepts a user's query.
- Process: It performs a vector similarity search within the associated store.
- Output: It returns the top k most similar document chunks.

Its primary function is to filter the massive database down to just the few specific paragraphs needed to answer the current question.

N° 29

QUALITY CONTROL

Why should you inspect the documents returned by the retriever?

THE ANSWER

Inspecting retrieved documents is a quality control step. Before passing data to the LLM, you should verify that:

1. The documents are actually relevant to the question.
2. The text is readable and not garbled code.

If the retriever is bringing back irrelevant context, the LLM will fail. Checking this output helps developers tune the chunk size or swap the embedding model to improve accuracy.

N° 30

RAG INTEGRATION

What is the goal of integrating retrieval with generation?

THE ANSWER

Integration combines the "Researcher" (Retrieval) with the "Writer" (Generation).

- Without integration: You have a search engine that gives you links, or a chatbot that hallucinates facts.
- With integration: The system finds the facts first, then uses the LLM to synthesize those facts into a coherent, human-like answer.

This fusion creates a system that is both factually grounded (thanks to retrieval) and conversational (thanks to generation).

SAVE · SHARE · SHIP

Found this *helpful?*

These 30 questions cover the entire RAG pipeline end to end. Share them with someone prepping for their next AI engineering interview.

DO THIS NEXT

01 Like this post so others find it.

02 Repost to help your network prep.

03 Follow JayTech for more AI deep dives.

Follow JayTech AI →

Comment "RAG" for the full guide